# TRANSFORMERS

Architecture, Tokenization, Embeddings,
Pre-training and Fine-tuning

Report by *Vivek Sharma*

# Outline

- *Transformer Architecture Basics*
- *Pretraining vs Fine tuning*
- *Tokenization*
- *Embeddings*

# What does a Transformer Do?

A Transformer is a neural-network architecture that processes sequences using attention rather than recurrence. Its central operation, self-attention, lets each token build a context-aware representation by weighting information from other tokens. A typical language-model pipeline converts text into token IDs, maps IDs to vectors, adds information about token positions, processes the vectors through stacked Transformer blocks, and converts the final representations into task-specific predictions or next-token probabilities.

- **Architecture at a glance**

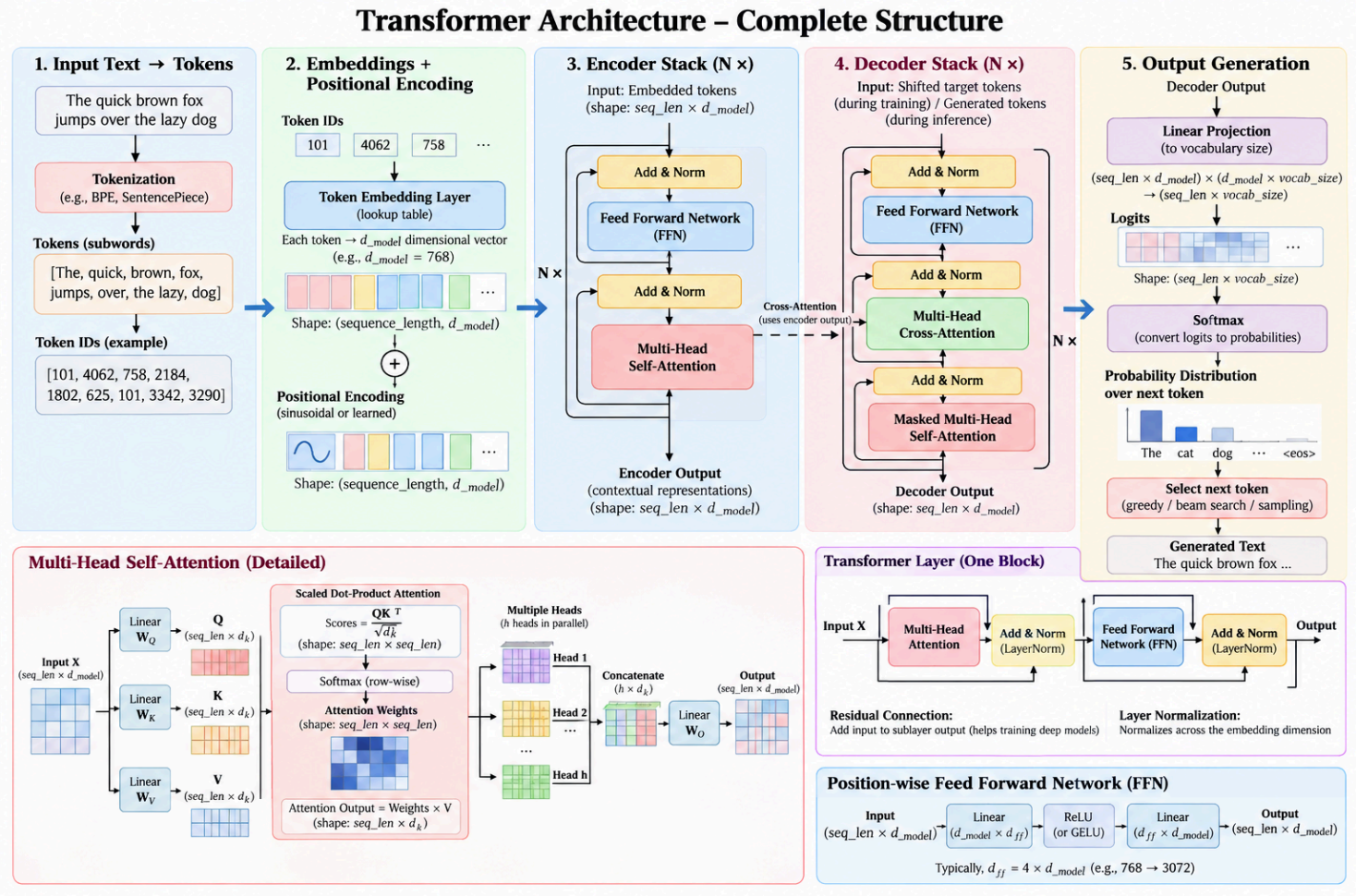
The original Transformer is an encoder–decoder model. Many modern language models use only one side of this design.

| Component | Role | Key idea |
|---|---|---|
| Input tokenizer | Converts text into token IDs | Text becomes a sequence of discrete symbols |
| Token embedding | Maps each ID to a vector | Learned numeric representation |
| Positional information | Signals token order | Attention alone does not inherently encode sequence order |
| Encoder stack | Builds contextual input representations | Bidirectional self-attention in the original design |
| Decoder stack | Generates output sequence | Causal self-attention plus encoder–decoder attention |
| Output projection + softmax | Produces scores/probabilities | Selects a token or task-specific prediction |

- **Three common Transformer configurations**

| Family | Attention pattern | Typical applications | Example |
|---|---|---|---|
| Encoder-only | Input tokens can attend to tokens on both sides | Classification, embeddings, entity extraction | BERT |
| Decoder-only | Causal: each position sees only earlier/current tokens | Text generation and chat | GPT-style models |
| Encoder–decoder | Encoder reads input; decoder generates output and attends to encoder | Translation, summarization, text transformation | Original Transformer; T5-style models |

These are common patterns rather than rigid rules. Modern systems may add retrieval, tools, adapters, multimodal encoders, or other components around the Transformer.




# Tokenization and embeddings

- **Tokenization: text to IDs**

A tokenizer splits text into units from a fixed vocabulary. A token can be a whole word, part of a word, punctuation, a byte-level unit, or a special control token. Subword methods (for example, BPE or WordPiece-style approaches) balance vocabulary size with the ability to represent rare or unseen words.

Illustrative example only (actual splits depend on the tokenizer):

**Text: “Transformers are useful.”**
Possible tokens: [“Transform”, “ers”, “ are”, “ useful”, “.”]
Possible token IDs: [812, 43, 289, 1902, 13]

The IDs above are invented to illustrate the process; they are not IDs from a real tokenizer. The model receives the IDs, not the original character string. Special tokens may mark sequence boundaries, padding, unknown items, or task instructions.

- **Embeddings: IDs to vectors**

Each token ID indexes a row in a learned embedding matrix. If the vocabulary has V tokens and the model width is d, the matrix is typically V × d. A token ID retrieves a d-dimensional vector. The vector values are learned during training; they are not hand-written definitions.

A crucial distinction: the initial token embedding is largely context-independent, while the hidden representation after attention layers becomes contextual. For example, “bank” in “river bank” and “bank account” starts from the same token embedding if the tokenizer uses the same token, but later layers can produce different representations based on context.

- **Positional information**

Self-attention needs information about order. The original Transformer adds sinusoidal positional encodings to token embeddings. Other models use learned positional embeddings or relative/rotary position methods. The chosen method affects how position and longer contexts are represented.

### Pipeline

**Raw text → tokenizer → token IDs → token embeddings + position information → Transformer layers**

# Pre-training versus fine-tuning

- **Pre-training**

Pre-training is the broad learning stage. The model is trained on a large and diverse corpus with a self-supervised objective, so labels can be derived from the text itself. A decoder-only model commonly learns to predict the next token. BERT-style encoder models traditionally use masked-language modeling, predicting selected masked tokens from surrounding context. The objective depends on the model family.

Purpose: learn reusable language patterns, representations, and statistical regularities.

Data: usually large-scale, diverse text (and, for multimodal models, possibly image, audio, or other data).

Cost: typically high compute, data, and engineering requirements.

Outcome: a base/pre-trained model that can be prompted or adapted to downstream tasks.

- **Fine-tuning**

Fine-tuning continues training from pre-trained weights using examples targeted at a task or behavior. For example, a general model may be fine-tuned on labeled customer-support messages, domain-specific question-answer pairs, or instruction-response examples. Fine-tuning can update all model weights or only a small subset.

Supervised fine-tuning (SFT): trains on example inputs and desired outputs.

Parameter-efficient fine-tuning (PEFT): updates a small number of added or selected parameters instead of all weights.

LoRA: a popular PEFT method that learns low-rank weight updates while keeping most original weights frozen.

Preference/alignment stages: some systems use preference data and methods such as reward modeling or direct preference optimization; these are related adaptation stages, not synonyms for ordinary SFT.

- **Comparison**

| Dimension | Pre-training | Fine-tuning |
| --- | --- | --- |
| Starting point | Usually random or specially initialized weights | A pre-trained checkpoint |
| Main goal | Learn broad, reusable patterns | Adapt to a task, domain, format, or behavior |
| Dataset | Large and broad; often unlabeled text | Smaller and more targeted; often labeled or curated examples |
| Objective | Self-supervised objective such as next-token prediction | Task loss, supervised next-token loss, or adaptation-specific objective |
| Compute | Usually very expensive | Usually less expensive, though large-model tuning can still be costly |
| Output | Base model | Task-adapted model or adapter |

Example: pre-training teaches a model general language patterns from broad text. Fine-tuning on a carefully prepared set of company policy questions and verified answers can make it more consistent at answering that kind of question. Fine-tuning does not guarantee factuality, and it is not a substitute for retrieval when answers must reflect frequently changing documents.